Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №5
з дисципліни «Проектування програмних систем для мобільних пристроїв»

«Архітектура додатків та допоміжні бібліотеки»

Варіант №4

Виконав:
студент 2 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Нестерук А. О.

Дата здачі: 21.10.2026

КИЇВ – 2026

Мета. Вивчити особливості архітектури програм для операційної системи Android та виділити їх як для платформи розробки програм.

Завдання. Додати у програму, розроблену у лабораторній роботі №4, сторонні бібліотеки, різними методами. Зробіть висновок про зручність цих методів. Скористайтеся бібліотеками з відкритих джерел Інтернету.

Завдання за варіантом. Доменна область: Абітурієнт.

Поля: ID, Прізвище, Ім'я, По-батькові, Адреса, Телефон, Оцінки.

Створити масив об'єктів.

Вивести:

а. список абітурієнтів, які мають незадовільні оцінки;

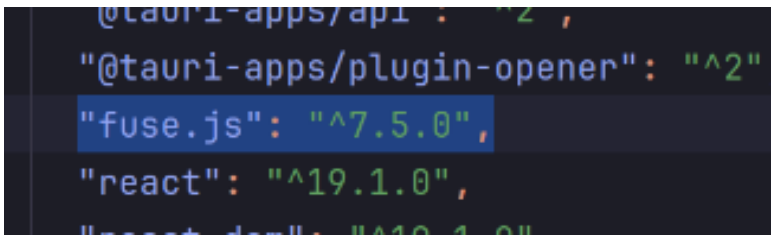
б. список абітурієнтів, середній бал у яких вище заданого;

в. вибрати задане число `N` абітурієнтів, що мають найбільш високій середній бал (вивести також повний список абітурієнтів, які мають напівпрохідний бал).

Хід роботи.

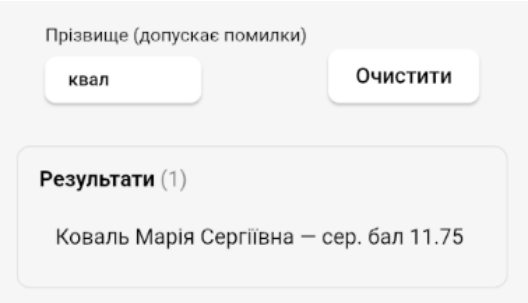
Для виконання роботи використано застосунок, розроблений у четвертій лабораторній роботі, без зміни архітектури MVC чи стеку – Rust, Tauri, React з TypeScript. Завдання вимагає додати до застосунку сторонні бібліотеки різними методами та зробити висновок про зручність цих методів; для порівняння обрано два принципово різні методи – підключення через реєстр пакетів (pnpm) та підключення через git-залежність (не crates.io).

Першим методом додано бібліотеку `fuse.js` – звичайним способом, командою `pnpm add fuse.js`, яка резолвить пакет через реєстр npm, записує точну версію в `package.json` і фіксує її разом з хешем цілісності в lock-файлі.



Бібліотеку використано на екрані «Пошук»: замість попереднього точного пошуку підрядка на боці Rust, екран тепер один раз завантажує повний список абітурієнтів і виконує нечіткий (typo-tolerant) пошук на клієнті за допомогою індексу `Fuse`, побудованого з полем `last_name` та порогом нечіткості `threshold: 0.4`. Фільтрація відбувається одразу при зміні поля вводу, без окремої команди на

сервер. Коректність перевірено на емуляторі: прізвище введено з однією помилкою (одна літера), і застосунок все одно знайшов відповідного абітурієнта.

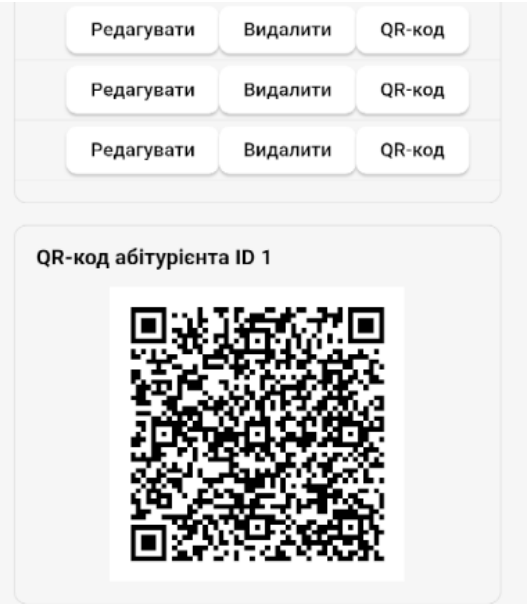


Другим методом додано крейт `qrcode` – через git-залежність у `Cargo.toml`: замість номера версії з crates.io вказано посилання на GitHub-репозиторій та тег `v0.14.1`. На відміну від звичайних залежностей, Cargo в цьому випадку клонує репозиторій безпосередньо за вказаним тегом, а не отримує пакет з реєстру crates.io.

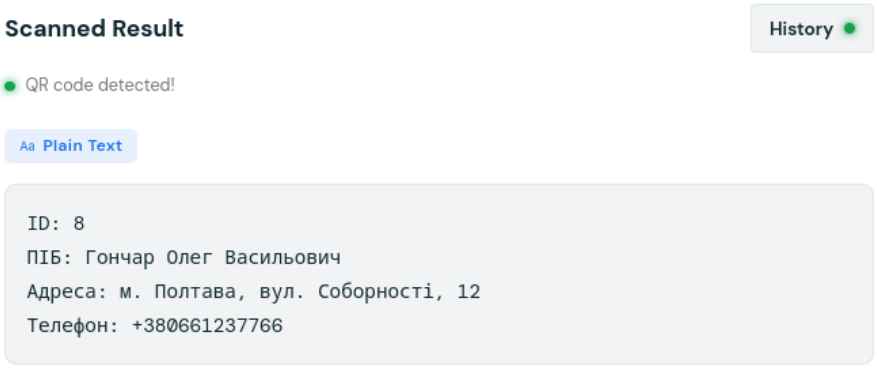
```
serde = { version = "1", features = ["derive"] }
serde_json = "1"

# Added via a git dependency (tag), not crates.io — demonstrates a second
# method of pulling in a third-party library, for comparison in the report.
qrcode = { git = "https://github.com/kennytm/qrcode-rust", tag = "v0.14.1", default-features = false, features = [
    "svg",
] }
```

На боці Rust додано контролер-модуль із відповідною командою, яка знаходить абітурієнта за `ID`, формує з його даних (ПІБ, адреса, телефон) текстовий рядок і кодує його у QR-код, після чого рендерить його в SVG-розмітку. На боці React додано кнопку «QR-код» у кожному рядку списку абітурієнтів на екрані «Абітурієнти». За натисканням під таблицею з’являється панель з отриманим SVG.



Коректність згенерованого QR-коду перевірено, зчитавши його через сайт – закодований текст відповідав даним вибраного абітурієнта.



Висновок. У цій лабораторній роботі до застосунку «Абітурієнт» додано дві сторонні бібліотеки різними шляхами.

Порівнюючи зручність методів, підключення через реєстр пакетів виявилося значно простішим і надійнішим: одна команда знаходить пакет, фіксує точну версію й хеш цілісності в lock-файлі, а оновлення версії так само зводиться до однієї команди; при цьому пакет пройшов публікацію в реєстрі, що дає певну гарантію цілісності та доступності. Підключення через git-залежність вимагає більше ручної роботи й уважності: Cargo не має семантичного резолвера версій для таких залежностей, тому без явного зазначення тегу чи коміту збірка була б нестабільною – залежність могла б непомітно змінитися разом зі змінами в репозиторії. З іншого боку, цей метод єдиний застосовний, коли бібліотека не опублікована в crates.io взагалі, або коли потрібна саме версія з конкретної гілки чи форку. Відповідно, реєстр пакетів варто використовувати як основний спосіб, а git-залежність – лише тоді, коли реєстр не надає потрібного варіанта бібліотеки.

Контрольні запитання.

1. **Охарактеризуйте архітектуру Android-програми.** Архітектура Android-програми базується на чотирьох фундаментальних компонентах: активностях (Activities) для візуальної взаємодії з користувачем, фонових службах (Services) для виконання тривалих операцій без графічного інтерфейсу, приймачах широкомовних повідомлень (Broadcast Receivers) для реакції на загальносистемні події та контент-провайдерах (Content Providers) для управління доступом та безпечного обміну даними між різними додатками. Сучасний підхід до розробки від Google також вимагає дотримання патерну MVVM (Model-View-ViewModel), де архітектура чітко розділяється на рівень інтерфейсу (View), рівень управління станом (ViewModel) та рівень

даних із репозиторіями, що забезпечує стабільність роботи при зміні конфігурації екрану та спрощує тестування коду.

2. **Назвіть класи-помічники.** Класи-помічники (Helper classes або Utility classes) у розробці під Android – це спеціальні класи, які інкапсулюють рутинну, повторювану логіку або специфічні алгоритми для розвантаження основних компонентів програми. Класичним системним прикладом є `SQLiteOpenHelper`, який бере на себе всю роботу зі створення, відкриття та управління версіями локальної бази даних, але до цієї категорії також належать будь-які власні класи зі статичними методами для валідації введених даних, форматування дат, конвертації величин, роботи з мережею або обробки зображень, які не прив'язані до життєвого циклу активностей чи фрагментів.
3. **Назвіть основні етапи підключення сторонніх бібліотек у проект.** Процес інтеграції розпочинається зі знаходження необхідної залежності в публічних репозиторіях на кшталт Maven Central, після чого розробник відкриває конфігураційний файл `build.gradle` (на рівні модуля додатка) і додає рядок із вказівкою групи, назви та версії інструменту у спеціальний блок dependencies. Завершальним і найважливішим етапом є натискання кнопки «Sync Now» в Android Studio, що запускає процес синхронізації Gradle, під час якого система автоматично завантажує всі потрібні файли, перевіряє сумісність версій та робить нові класи доступними для використання в основному коді проекту.

Лістинг.

qr.rs

```rust
//! Lab 5: Generating a QR code containing applicant data using the `qrcode` crate,
//! integrated via a Git dependency (rather than crates.io) — a demonstration of an alternative
//! method for adding third-party code to a project.

// State<T> must be taken by value in every command here — that's Tauri's
// calling convention, not a real ownership need.
#![allow(clippy::needless_pass_by_value, reason = "Tauri's convention")]

use crate::repository::Repository;
use qrcode::QrCode;
use qrcode::render::svg;
use std::sync::{Mutex, PoisonError};
use tauri::State;

#[tauri::command]
pub fn generate_qr(
state: State<'_, Mutex<Repository>>, id: u32,
) -> Result<String, String> {
    let (last_name, first_name, patronymic, address, phone) = state
    .lock()
    .unwrap_or_else(PoisonError::into_inner)
    .list()
    .iter()
    .find(|a| a.id == id)
    .map(|a| {
        (
        a.last_name.clone(),
        a.first_name.clone(),
        a.patronymic.clone(),
        a.address.clone(),
        a.phone.clone(),
        )
    })
    .ok_or_else(|| format!("Абітурієнта з ID {id} не знайдено"))?;

    let payload = format!(
    "ID: {id}\nПІБ: {last_name} {first_name} {patronymic}\nАдреса: {address}\nТелефон: {phone}",
    );

    let code = QrCode::new(payload.as_bytes()).map_err(|err| err.to_string())?;

    Ok(code.render::<svg::Color>().build())
}
```

Search.tsx

```tsx
import { useEffect, useMemo, useState } from "react";
import { invoke } from "@tauri-apps/api/core";
import { Link } from "react-router-dom";
import Fuse from "fuse.js";
import type { Applicant } from "../types";

// Demonstrates Button, TextView, EditText, and ListView on one screen
// (Lab 4 assignment). Lab 5 adds fuse.js here (via the pnpm/npm registry)
// for typo-tolerant fuzzy matching on last names — the other new library
// this lab adds, the `csv` crate on the Students screen, is pulled in via
// a git dependency instead, to compare the two installation methods.
export function Search() {
```

```
    const [query, setQuery] = useState("");
    const [all, setAll] = useState<Applicant[] | null>(null);
    const [selectedId, setSelectedId] = useState<number | null>(null);

    useEffect(() => {
        invoke<Applicant[]>("list_applicants").then(setAll);
    }, []);

    const fuse = useMemo(
    () => new Fuse(all ?? [], { keys: ["last_name"], threshold: 0.4 }),
    [all],
    );

    const rows = useMemo(() => {
        if (!all) {
            return null;
        }
        if (query.trim() === "") {
            return all;
        }
        return fuse.search(query).map((result) => result.item);
    }, [all, query, fuse]);

    function clear() {
        setQuery("");
        setSelectedId(null);
    }

    const selected = rows?.find((a) => a.id === selectedId) ?? null;

    return (
    <main className="container">
    <Link className="back" to="/">
    ← На головну
    </Link>
    <h1>Пошук абітурієнтів</h1>
    <p className="subtitle">
    EditText + Button + ListView + TextView, нечіткий пошук через fuse.js
    </p>

    {/* EditText + Button */}
    <form className="controls" onSubmit={(e) => e.preventDefault()}>
    <label>
    Прізвище (допускає помилки)
    <input
    type="text"
    value={query}
    onChange={(e) => setQuery(e.currentTarget.value)}
    />
    </label>
    <button type="button" onClick={clear}>
    Очистити
    </button>
    </form>

    {/* ListView */}
    <section className="panel">
    <h2>
    Результати <span className="count">({rows?.length ?? 0})</span>
    </h2>
    {rows && rows.length > 0 ? (
        <ul className="list-view">
        {rows.map((a) => (
            <li
```

```
            key={a.id}
            className={a.id === selectedId ? "selected" : undefined}
            onClick={() => setSelectedId(a.id)}
            >
            {a.last_name} {a.first_name} {a.patronymic} — сер. бал{" "}
            {a.average}
            </li>
            ))}
        </ul>
        ) : (
        <p className="empty">— немає —</p>
        )}
    </section>

    {/* TextView */}
    <section className="panel">
    <h2>Деталі</h2>
    {selected ? (
        <p className="text-view">
        ID: {selected.id}
        <br />
        ПІБ: {selected.last_name} {selected.first_name}{" "}
        {selected.patronymic}
        <br />
        Адреса: {selected.address}
        <br />
        Телефон: {selected.phone}
        <br />
        Оцінки: {selected.grades.join(", ")}
        <br />
        Середній бал: {selected.average}
        </p>
        ) : (
        <p className="empty">Виберіть абітурієнта зі списку</p>
        )}
    </section>
    </main>
    );
}
```

Students.tsx

```
import { useEffect, useState, type FormEvent } from "react";
import { invoke } from "@tauri-apps/api/core";
import { Link } from "react-router-dom";
import type { Applicant, ApplicantInput } from "../types";

const emptyForm = {
    last_name: "",
    first_name: "",
    patronymic: "",
    address: "",
    phone: "",
    gradesText: "",
};

function parseGrades(text: string): number[] {
    return text
    .split(/[,\s]+/)
    .map((token) => Number(token))
    .filter((n) => Number.isFinite(n) && n >= 0);
}

```

```tsx
export function Students() {
    const [rows, setRows] = useState<Applicant[] | null>(null);
    const [editingId, setEditingId] = useState<number | null>(null);
    const [form, setForm] = useState(emptyForm);
    const [error, setError] = useState<string | null>(null);
    const [qrFor, setQrFor] = useState<number | null>(null);
    const [qrSvg, setQrSvg] = useState<string | null>(null);

    useEffect(() => {
        invoke<Applicant[]>("list_applicants").then(setRows);
    }, []);

    function startEdit(a: Applicant) {
        setEditingId(a.id);
        setForm({
            last_name: a.last_name,
            first_name: a.first_name,
            patronymic: a.patronymic,
            address: a.address,
            phone: a.phone,
            gradesText: a.grades.join(", "),
        });
        setError(null);
    }

    function cancelEdit() {
        setEditingId(null);
        setForm(emptyForm);
        setError(null);
    }

    async function showQr(id: number) {
        if (qrFor === id) {
            setQrFor(null);
            setQrSvg(null);
            return;
        }

        try {
            const svg = await invoke<string>("generate_qr", { id });
            setQrFor(id);
            setQrSvg(svg);
        } catch (err) {
            setError(String(err));
        }
    }

    async function remove(id: number) {
        if (!window.confirm(`Видалити абітурієнта з ID ${id}?`)) {
            return;
        }

        try {
            await invoke("delete_applicant", { id });
            setRows((prev) => (prev ? prev.filter((a) => a.id !== id) : prev));
            if (editingId === id) {
                cancelEdit();
            }
        } catch (err) {
            setError(String(err));
        }
    }

    async function submit(e: FormEvent) {
```

```
        e.preventDefault();
        const input: ApplicantInput = {
            last_name: form.last_name,
            first_name: form.first_name,
            patronymic: form.patronymic,
            address: form.address,
            phone: form.phone,
            grades: parseGrades(form.gradesText),
        };

        try {
            if (editingId === null) {
                const created = await invoke<Applicant>("add_applicant", { input });
                setRows((prev) => (prev ? [...prev, created] : [created]));
            } else {
                const updated = await invoke<Applicant>("update_applicant", {
                    id: editingId,
                    input,
                });
                setRows((prev) =>
                prev ? prev.map((a) => (a.id === editingId ? updated : a)) : prev,
                );
            }
            cancelEdit();
        } catch (err) {
            setError(String(err));
        }
    }

    return (
    <main className="container">
    <Link className="back" to="/">
    ← На головну
    </Link>
    <h1>Абітурієнти</h1>

    <form className="form" onSubmit={submit}>
    <h2>{editingId === null ? "Додати абітурієнта" : `Редагувати ID ${editingId}`}</h2>
    {error && <p className="error">{error}</p>}
    <label>
    Прізвище
    <input
    required
    value={form.last_name}
    onChange={(e) => setForm({ ...form, last_name: e.currentTarget.value })}
    />
    </label>
    <label>
    Ім'я
    <input
    required
    value={form.first_name}
    onChange={(e) => setForm({ ...form, first_name: e.currentTarget.value })}
    />
    </label>
    <label>
    По-батькові
    <input
    value={form.patronymic}
    onChange={(e) => setForm({ ...form, patronymic: e.currentTarget.value })}
    />
    </label>
    <label>
    Адреса
```

```
        <input
        value={form.address}
        onChange={(e) => setForm({ ...form, address: e.currentTarget.value })}
        />
        </label>
        <label>
        Телефон
        <input
        value={form.phone}
        onChange={(e) => setForm({ ...form, phone: e.currentTarget.value })}
        />
        </label>
        <label>
        Оцінки (через кому)
        <input
        placeholder="10, 11, 9, 12"
        value={form.gradesText}
        onChange={(e) => setForm({ ...form, gradesText: e.currentTarget.value })}
        />
        </label>
        <div className="form-actions">
        <button type="submit">{editingId === null ? "Додати" : "Зберегти"}</button>
        {editingId !== null && (
            <button type="button" onClick={cancelEdit}>
            Скасувати
            </button>
            )}
        </div>
        </form>

        <section className="panel">
        <h2>
        Список <span className="count">({rows?.length ?? 0})</span>
        </h2>
        {rows && rows.length > 0 ? (
            <table>
            <thead>
            <tr>
            <th>ID</th>
            <th>ПІБ</th>
            <th>Адреса</th>
            <th>Телефон</th>
            <th>Оцінки</th>
            <th>Сер. бал</th>
            <th></th>
            </tr>
            </thead>
            <tbody>
            {rows.map((a) => (
                <tr key={a.id}>
                <td>{a.id}</td>
                <td>
                {a.last_name} {a.first_name} {a.patronymic}
                </td>
                <td>{a.address}</td>
                <td>{a.phone}</td>
                <td>{a.grades.join(", ")}</td>
                <td>{a.average}</td>
                <td>
                <button type="button" onClick={() => startEdit(a)}>
                Редагувати
                </button>
                <button type="button" onClick={() => remove(a.id)}>
                Видалити
```

```
            </button>
            <button type="button" onClick={() => showQr(a.id)}>
            {qrFor === a.id ? "Сховати QR" : "QR-код"}
            </button>
            </td>
            </tr>
            ))}
        </tbody>
        </table>
        ) : (
        <p className="empty">Поки немає заявок</p>
        )}
    </section>

    {qrFor !== null && qrSvg && (
        <section className="panel">
        <h2>QR-код абітурієнта ID {qrFor}</h2>
        <div className="qr-code" dangerouslySetInnerHTML={{ __html: qrSvg }} />
        </section>
        )}
    </main>
    );
}
```